Neural Networks 201 (2026) 108901

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

DAG-NAS : An explainable neural architecture search framework for reinforcement learning

Taegun An, Changhee Joo *

Department of Computer Science and Engineering, Korea University, Seoul 02841, Republic of Korea

ARTICLE INFO

Keywords:
Neural architecture search
Reinforcement learning
Directed acyclic graph
Explainable architecture search

ABSTRACT

We present an explainable Neural Architecture Search (NAS) framework for Reinforcement Learning (RL). We model a feed-forward neural network as a Directed Acyclic Graph (DAG) that consists of *scalar-level* operations and their interconnections. Scalar-level DAGs are trained using a differentiable search method, followed by pruning search results. This approach yields a compact neural architecture that delivers high performance while enhancing explainability by highlighting the critical information needed to solve the problem. We apply our NAS framework to search both actor and critic networks of the Actor-Critic PPO algorithm across various RL tasks. Extensive experiments demonstrate that our architectures achieve comparable performance with significantly fewer parameters while highlighting key features and enhancing explainability.

1. Introduction

Machine learning, a subset of artificial intelligence, has undergone a huge evolution driven by the advent of deep learning. A large number of deep learning methods hinge on effective neural network architectures, which are typically crafted by human experts through a time-consuming and often intuition-based process. Neural Architecture Search (NAS) has made a significant paradigm shift by automating the design of neural network architectures. Since the seminal work of Zoph and Le (2017), extensive research in NAS has systematically explored vast architecture spaces, discovering novel and high-performing network designs across various domains like Natural Language Processing (NLP), Computer Vision (CV), and Reinforcement Learning (RL) (Kang et al., 2023; Klyuchnikov et al., 2020; Parker-Holder et al., 2022a). Among these, NAS for RL has been relatively less explored due to its unique challenges, such as fragile RL algorithms, high computational costs, and hyperparameter sensitivity, while holding great potential to expand the boundaries of what machine learning can accomplish (White et al., 2023).

Despite its considerable potential, NAS encounters various challenges and limitations. One of the major challenges arises from the need for expertise in designing the search space for NAS, which results in a trade-off between the investment in the search space design and the search complexity. In addition, an overly customized search space aimed at optimizing architecture may hinder the exploration of potentially better architectures and introduce bias into the search process (Liu et al., 2018). It has been reported that in such overly constrained search spaces, even random search performs equally well (Li & Talwalkar, 2020).

Another important challenge is the insufficient explainability of the searched architectures. It is hard to understand how they make their decisions and what information is crucial for it. This issue mirrors a broader challenge in deep learning, where the intricate network structure often results in a black-box model with limited insight into its decision-making process. In deep learning, there have been several studies to improve the model explainability (Lundberg & Lee, 2017; Montavon et al., 2017), which, however, often necessitate several assumptions or are constrained to specific models. In NAS, there have also been continuing efforts to enhance the explainability of NAS-generated models (Kadra et al., 2023), yet there is still much room for improvement. Moreover, in practical RL problems, the agent has to solve sequential decision-making problems given the system state as input. This state information often includes several measurements from various sensors that may be arbitrarily correlated. In this case, model explainability is crucial for identifying key features needed to solve the problem, which contributes to reducing the model size.

A key motivation of this work is that state representations in reinforcement learning are often overcomplete and highly correlated. Conventional neural architectures and NAS methods primarily focus on performance, and do not reveal which state variables drive decision making. This limits their practical utility for understanding, debugging, and simplifying RL policies. In this work, we model neural networks as scalar-level directed acyclic graphs, enabling finer-grained architecture

* Corresponding author.
E-mail addresses: antaegun20@korea.ac.kr (T. An), changhee@korea.ac.kr (C. Joo).

https://doi.org/10.1016/j.neunet.2026.108901
Received 15 August 2025; Received in revised form 4 March 2026; Accepted 25 March 2026
Available online 3 April 2026

search than traditional layer-level or cell-level NAS. This design enables the search process to implicitly perform input-level feature selection, yielding compact architectures with directly interpretable decision pathways. In this way, it bridges performance-oriented NAS and explainable reinforcement learning.

In this work, we introduce a novel NAS approach that enhances search flexibility and offers high explainability through high-precision DAG modeling. We decompose a typical feed-forward neural network into scalar-level operations using single-output vertices. By relaxing discrete requirements on the operations and interconnections, we employ a fully-differentiable architecture search. The resulting architectures are then discretized and pruned to achieve a lightweight and high-performing design. We demonstrate the effectiveness of our approach across various RL tasks.

Our contributions can be summarized as follows.

- To the best of our knowledge, we are the first to introduce scalar-level DAGs for modeling neural networks in architecture search. Our method reduces the effort required for designing the search space, enhances search flexibility, and offers high explainability.
- We have developed an effective framework of our DAG-NAS for searching and selecting neural architectures. (i) We extend constraint relaxation to both vertices and edges in DAGs, making a fully differentiable search process applicable. (ii) We introduce a correlation-based pruning to achieve a compact architecture with high explainability.
- We evaluate our framework across a broad spectrum of RL tasks and demonstrate that the architectures discovered by our framework have significantly fewer parameters while achieving comparable performance.

2. Related works

Neural Architecture Search (NAS) aims to automate neural network design and is typically characterized by three components: the search space, the search strategy, and the performance estimation strategy (Elsken et al., 2019). The search space defines the set of candidate architectures, the search strategy determines how these candidates are explored, and the performance estimation strategy evaluates their quality. For example, the seminal work in Zoph and Le (2017) employed a cell-based search space and an RL-based search strategy, where architectures were rewarded based on validation accuracy.

An important and common challenge of NAS is the search space design. It should be sufficiently large to enclose high-performing architectures, and at the same time, carefully constrained for efficient search with feasible computational complexity. To balance this trade-off, many works adopt structured search spaces such as chain, cell, or hierarchical designs (White et al., 2023). Extensions including dynamic search spaces (Ci et al., 2021; Xia et al., 2022), evolving spaces (Zhou et al., 2021), and expanded search formulations (Geada & McGough, 2022) aim to increase flexibility. However, these approaches still rely on predefined structural units (e.g., cells or blocks), which require manual design choices and limit architectural granularity.

Another important challenge is interpretability of the searched architectures. Understanding why a discovered architecture performs well remains difficult, particularly when complex cell-based structures are used. This issue is especially relevant in NAS-for-RL, where identifying task-relevant state features can provide practical insights and reduce model complexity. Prior efforts to improve interpretability include motif-based analysis (Ru et al., 2021), evolutionary approaches examining input–output relations (Carmichael et al., 2023), and DAG-based cell modeling (Lee et al., 2021; White et al., 2020). While these approaches enhance structural transparency to some extent, they often operate at the level of cells or modules, limiting fine-grained feature-level interpretation.

NAS for RL introduces additional challenges beyond those in supervised learning, including sensitivity to learning dynamics, reward variance, and algorithmic stability (Parker-Holder et al., 2022b). Compared to supervised NAS, relatively fewer works explicitly target architecture design in RL settings (Franke et al., 2021; Mysore et al., 2021). A representative example is Weight Agnostic Neural Networks (WANN) (Gaier & Ha, 2019), which explores architectures independently of trained weights using evolutionary strategies. Although WANN highlights the importance of structural design in RL, it incurs substantial computational cost and often produces complex architectures with many parameters.

Motivated by these limitations, we focus on a finer-grained architectural representation that reduces manual structural assumptions while enabling explicit feature-level connectivity modeling. By formulating neural networks as scalar-level directed acyclic graphs, we enlarge the architectural search space in a controlled manner and allow the search process to directly identify task-relevant state variables. This design reduces human intervention in architecture specification while promoting compactness and interpretability in reinforcement learning settings.

3. Preliminaries

RL is deeply rooted in the mathematical framework of Markov Decision Processes (MDPs), which provide a structured way of modeling an environment. An MDP is formally defined as a tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition probability function, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. Given state $S_t = s$ at time t, an agent takes action $A_t = a$ and the state transits to state $S_{t+1} = s'$ at time $t + 1$ with probability $T(s, a, s') = \Pr(S_{t+1} = s' | S_t = s, A_t = a)$, and the agent obtains reward R_{t+1}.

The agent learns to make decisions through trial-and-error interactions with the environment. The goal of the RL agent is to find an optimal policy $\pi : \mathcal{S} \to \mathcal{A}$ that maximizes the expected reward sum $\mathbb{E}[G_t]$ where $G_t = \sum_{k=t}^{\infty} \gamma^{k-t} R_{k+1}$. With the advance of deep learning, neural networks have been employed as an approximate decision-making function, replacing the agent. This technique, known as Deep RL (DRL), can be divided into two categories, value-based and policy-based methods. In the value-based method, neural models estimate the value of states and actions, and in the policy-based method, often referred to as the policy gradient method, they directly yield an action for a given state.

PPO is one of the most popular policy gradient methods in DRL. It utilizes two neural networks: one for the policy (actor) and the other for value estimation (critic). Their weights are adjusted based on the gradients of the following loss function:

$$\mathcal{L}(w) = \mathbb{E}_t[\min(r_t(w) \cdot \hat{A}_t,\ clip(r_t(w), \epsilon) \cdot \hat{A}_t], \tag{1}$$

where w denotes the network weight, $r_t(w) = \frac{\pi_w(a_t|s_t)}{\pi_{w_{old}}(a_t|s_t)}$ is the probability ratio for the sampling with current state s_t and action a_t, $clip(a, b) = \max\{1 - b, \min\{a, 1 + b\}\}$, $\hat{A}_t$ is an estimator of the advantage function defined as $\mathbb{E}[G_t|s_t, a_t] - \mathbb{E}[G_t|s_t]$. PPO maintains a relatively small deviation from the previous policy (w_{old}), ensuring training stability and reducing sensitivity to hyperparameters. PPO has shown remarkable performance in various domains, ranging from video games to robotic control, making it a popular baseline RL algorithm.

4. Methods

In this section, we describe the proposed DAG-NAS framework. We model neural networks as scalar-level directed acyclic graphs (DAGs) and construct a DAG supernetwork by relaxing discrete architectural constraints. Architecture search is performed using a differentiable optimization scheme, followed by discretization to obtain the final Discrete DAG (DDAG).

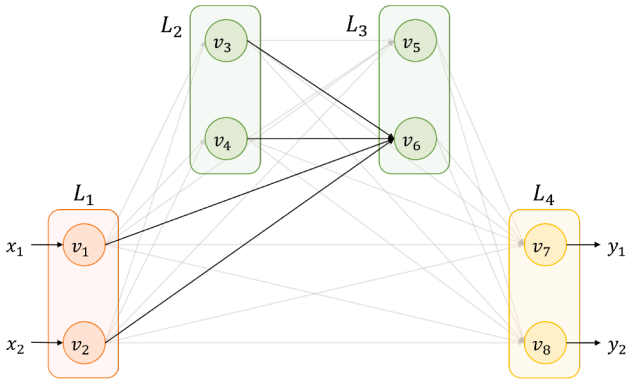

Fig. 1. Vertices and connections of DAG for a 4-layer neural network. The possible input connections to vertex v_6 are marked by dark arrows.

Previous NAS methods without considering selection of input features suffer from high complexity due to state inputs that are overcomplete, highly correlated, partially relevant or often harmful to decision making. We address this challenge by allowing connectivity decisions at the level of individual scalar features. Unlike layer- or cell-level DAGs that bundle multiple inputs into fixed computational blocks, scalar-level representations decouple feature selection from representation learning. This enables architecture search to explicitly determine which state variables influence policy decisions, yielding compact and interpretable architectures.

Since layer- or cell-level DAGs cannot express selective input pruning without additional mechanisms, scalar-level DAGs represent a qualitatively different search space rather than a simple refinement. This fine-grained control over connectivity naturally supports both interpretability and parameter efficiency in reinforcement learning tasks.

4.1. Search space of scalar-level DAG

We consider a multi-layer feed-forward network $f : \mathbb{R}^a \to \mathbb{R}^b$ with input $\mathbf{x} = (x_1, \dots, x_a)$ and output $\mathbf{y} = (y_1, \dots, y_b)$. It is represented as a graph $G(V, E)$ with the set V of vertices and the set E of directed edges or connections. A vertex collects outputs from incoming vertices and conducts an operation. The network can be divided into l layers, and a vertex belongs to one and only one layer. Let L_i denote the set of vertices in the ith layer. We assume that an edge can be connected from a vertex in L_i to a vertex in L_j, satisfying $i < j$. Thus, the graph is acyclic with one-directional data flow and has no edge between vertices in the same layer. Let $|\cdot|$ denote the set cardinality. For ease of exposition, we number the vertices following the data flow[1] in the forward path, i.e., $V = \{v_i\}_{i=1}^n$, where v_i is the ith vertex and $n = |V|$. A directed connection $v_i \to v_j$ is represented by (v_i, v_j), which must satisfy $i < j$ to align with the direction of data flow.

Note that DAG is not sufficient to identify a neural network model, and the same DAG can represent multiple models. To this end, we decompose a neural network model into its architecture, which is represented by a DAG, and weight parameters. Let c_{ij} denote the connectivity between vertices i and j, i.e., $c_{ij} = 1$ if $(v_i, v_j) \in E$, and $c_{ij} = 0$ otherwise. Given input $\mathbf{x}$, the output of v_j can be written as

$$v_j(\mathbf{x}) = o_j\left(\sum_{i=1}^{j-1} c_{ij} v_i(\mathbf{x}); \mathbf{w}_j\right), \tag{2}$$

where $o_j(\cdot; \mathbf{w}_j) : \mathbb{R} \to \mathbb{R}$ denotes the operation of vertex v_j, e.g., $LeakyReLU$. We will provide a more detailed description of the operations later. By *model architecture* or architecture, we refer to $\alpha = (\mathbf{o}, \mathbf{c})$ with $\mathbf{o} = \{o_i\}$ and $\mathbf{c} = \{c_{ij}\}$, and by *weight parameters* or weights, we refer to $\mathbf{w} = \{\mathbf{w}_i\}$. Certainly, this modeling can represent any feed-forward neural network, and given the architecture $\mathbf{o}, \mathbf{c}$ and weights $\mathbf{w}$, we can construct a neural network model. We emphasize that each vertex output is a scalar value, and connectivity is defined on a per-vertex basis rather than a per-layer basis. We refer to this structured DAG as a *scalar-level DAG*.

Algorithm 1 Forward computation of scalar-level DAG neural network.

Input: $\mathbf{x} = (x_1, \dots, x_a) \in \mathbb{R}^a$
Parameter: $\alpha = (\mathbf{o}, \mathbf{c}), \mathbf{w}$
Output: $\mathbf{y} = (y_1, \dots, y_b) \in \mathbb{R}^b$
$\mathbf{h}_1 \leftarrow \mathbf{x}$
for each $i = 2, \dots, l$ **do**
  $\mathbf{h}_i^{out} \leftarrow \{\}$
  for each vertex $v_j \in L_i$ **do**
    $\mathbf{h}_i^{out} \leftarrow concat(\mathbf{h}_i^{out}, o_j(\langle \mathbf{c}_j, \mathbf{h}_{i-1} \rangle))$
    $\mathbf{h}_i^{out} \leftarrow concat(\mathbf{h}_i^{out}, o_j(\langle \mathbf{c}_j, \mathbf{h}_{i-1} \rangle; \mathbf{w}_j))$
  end for
  $\mathbf{h}_{i+1} \leftarrow concat(\mathbf{h}_i, \mathbf{h}_i^{out})$
end for
return $\mathbf{y} \leftarrow \mathbf{h}_l^{out}$

Fig. 1 illustrates a 4-layer neural network with 2 vertices in each layer. Vertex $v_6 \in L_3$ can accept any outputs of vertices in L_1 and L_2, which are marked by dark arrows. This structure admits the representation of a general feed-forward network model with all possible skip connections. Let $\mathbf{h}_i^{out}$ denote the outputs of the vertices in L_i, i.e., $\mathbf{h}_i^{out} = \{v_i(\mathbf{x})\}_{i \in L_i}$, and let $\mathbf{h}_j$ as their concatenations up to $j-1$, satisfying $\mathbf{h}_j = concat(\mathbf{h}_{j-1}, \mathbf{h}_{j-1}^{out})$ and[2] $\mathbf{h}_1 = \mathbf{x}$. The input of the vertex operation o_j at layer L_k can be represented as

$$\sum_{i=1}^{j-1} c_{ij} v_i(\mathbf{x}) = \langle c_j, h_{k-1} \rangle, \tag{3}$$

where $\mathbf{c}_j = \{c_{ij}\}_{i=1}^{j-1}$ and $\langle \cdot, \cdot \rangle$ denote the inner product. Algorithm 1 describes the forward computation of an l-layer neural network, represented by architecture α and weights $\mathbf{w}$.

In this work, we assume that the number l of layers and the number of vertices at each layer are given, and focus on the search for the operation $\mathbf{o}$ and connectivity $\mathbf{c}$. Extension to the search for l and the number of vertices remains an interesting and important open problem.

4.2. Architecture search of scalar-level DAG

For the architecture search, we employ the well-known differential architecture search of DARTS (Liu et al., 2019). It admits gradient-based optimizations by relaxing the discrete architectural constraints and enables efficient exploration over a larger search space, becoming one of the most popular search strategies.

For the differentiable search, we replace binary connectivity $c \in \{0, 1\}$ with mixed connectivity $\bar{c} \in [0, 1]$, and denote vertex j's incoming mixed connectivity as $\bar{\mathbf{c}}_j = \{\bar{c}_{ij}\}_{v_i \in L_{<j}}$, where $L_{<j} = \cup_{i<j} L_i$. Similarly, we relax the discrete constraint of the vertex operation as follows. Suppose that we have the operation search space $\mathcal{O}$ that consists of $|\mathcal{O}|$ operations, i.e., $\mathcal{O} = \{o^1, \dots, o^{|\mathcal{O}|}\}$. For each vertex v_j, we introduce operation weights $\mathbf{a}_j = \{a_j^1, \dots, a_j^{|\mathcal{O}|}\}$ that satisfy $\sum_{k=1}^{|\mathcal{O}|} a_j^k = 1$ and $a_j^k \geq 0$ for all k, and define its mixed operation $\bar{o}_j$ as

$$\bar{o}_j(x) = \sum_{k=1}^{|\mathcal{O}|} a_j^k o^k(x). \tag{4}$$

In this work, we consider three candidate operations of $\mathcal{O} = \{Tanh, LeakyReLU, Linear\}$ for each vertex, where $Tanh(x) = \tanh(x)$,

[1] There is a tie if two vertices are parallel in the data flow, in which case we break the tie arbitrarily.

[2] The input layer L_1 is an exception, where the vertices are predetermined. Specifically, a vertex in L_1 accepts an element of input data $\mathbf{x}$ and passes itself as the output with no operation, yielding $\mathbf{h}_1 = \mathbf{x}$.

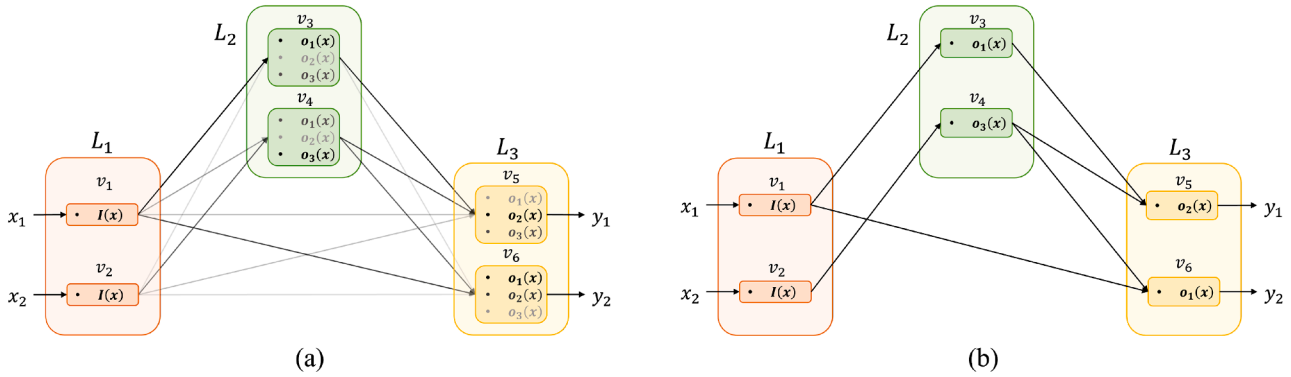

Fig. 2. After the differentiable architecture search, we obtain (a) a DAG supernetwork with mixed operations and connectivities. We represent the relative importance of operations and connections by the darkness of fonts and arrows, respectively. From the DAG supernetwork, we obtain the final architecture called (b) Discrete DAG (DDAG) through the discretization process that prunes less important operations and connections.

$LeakyReLU(x) = \max(0.2x, x)$, and $Linear(x) = w_1 x + w_2$. Note that $Linear$ has two trainable weights w_1 and w_2, while $Tanh$ and $LeakyReLU$ have no weights. The set $\mathcal{O}$ can be readily expanded to include additional options.

Through the above relaxation, we obtain a DAG supernetwork with mixed operations and connectivities. This allows us to apply the fully differential architecture search for the scalar-level DAG. During the learning procedure, we take the gradients of the loss function, and obtain optimal mixed variables $\bar{\alpha} = (\bar{\mathbf{o}}, \bar{\mathbf{c}})$. The details are as follows.

- We adopt the PPO algorithm for the learning, in which case the PPO objective (1) is used as the loss function.
- We take the single-level optimization approach to learn the architecture (α) and weights (w) together:

$$(\alpha^*, w^*) = \arg\min_{\alpha, w} \mathcal{L}(\alpha, w). \tag{5}$$

  The single-level optimization is computationally efficient due to fewer optimizations, thus used in NAS works with diverse optimization complexities (An & Joo, 2024; Xie et al., 2019). Due to the high complexity and instability of learning in RL tasks, reducing the number of optimization steps while matching the direction of optimization is important, and single-level optimization is advantageous in both perspectives.
- After the learning process completes, we store the final architecture $\bar{\alpha}^*$ and weights $\mathbf{w}^*$. Note that, unlike many previous NAS schemes, we do not store intermittent architectures and make use of only the last architecture. This implies that we remove the high-cost after-search evaluation process that most NAS schemes require to determine the final outcome among a number of candidate architectures.

The outcome $\bar{\alpha}^*$ is a supernetwork architecture with mixed variables, which need to be further discretized to obtain a feasible architecture. Fig. 2(a) shows an example of a DAG supernetwork after the differentiable architecture search, where the relative importance of operations and connectivities in mixed variables is represented by the darkness of the fonts and arrows, respectively.

Remark: The PPO algorithm may fail to achieve high rewards (Agarwal et al., 2021; Chan et al., 2020), in which case, a single search results in a low-performing architecture. In our work, we repeat the search 5 times and select the best outcome.

4.3. Architecture discretization

After the differential architecture search, we have a fully trained DAG supernetwork with learned connectivities and operation importances. However, often the DAG supernetwork contains redundant connections that convey overlapping information. We obtain a compact and interpretable discrete architecture by choosing one operation out of the candidate operations in each vertex, and by discretizing the connectivities.

We first identify information overlap between connected features by employing correlation measures, and then prune the fully trained DAG supernetwork using the correlation. After pruning, we finalize the architecture and obtain $\alpha^* = (\mathbf{o}^*, \mathbf{c}^*)$ from the search outcome $\bar{\alpha}^* = (\bar{\mathbf{o}}^*, \bar{\mathbf{c}}^*)$. The detailed steps are as follows.

For each vertex v_j, we select the operation with the largest importance, i.e.,

$$o_j^* = o^\kappa, \text{ where } \kappa = \arg\max_k a_j^k. \tag{6}$$

If the chosen operation includes learnable weights (e.g., $Linear$), we also use the weights from $\mathbf{w}^*$ without retraining.

For connectivity, we discretize the mixed connectivities based on the correlation between the vertex outputs as follows.

1. We generate a number of synthetic inputs $\mathbf{x}$ by sampling uniformly from $[-1, 1]^a$, where a is the input dimension.
2. We forward the synthetic inputs $\{\mathbf{x}\}$ in to the model, and compute the mutual correlations between the vertex outputs *within the same layer*. We partition[3] the vertices such that all the vertices in the same group have a mutual correlation higher than 0.9. In each group, we maintain one vertex (arbitrarily chosen) and remove all the other vertices from the supernetwork.
3. Then, we compute the correlation between vertices i, j *across layers* and discretize the connectivity using a threshold of 0.5, i.e., $c_{ij}^* = 1$ if the correlation is greater than 0.5 and $c_{ij}^* = 0$ otherwise.

Note that Step 2 merges similar vertices into a single node, and Step 3 prunes less significant connections.

After the discretization process, we obtain a Discrete DAG architecture, or simply DDAG. An example is shown in Fig. 2(b). Once DDAG is determined, we keep the trained weights $\mathbf{w}$ from the supernetwork and omit the typical weight-retraining process of NAS. DDAG has significantly fewer parameters while still achieving good performance.

This pruning process serves two complementary purposes. First, it substantially reduces the number of parameters, yielding a compact Discrete DAG (DDAG) without retraining network weights. Second, by eliminating redundant information paths, pruning simplifies the connectivity structure and enhances interpretability, making the contribution of individual state variables to policy decisions more explicit. As a result, the pruning strategy simultaneously improves architectural efficiency and explainability, which is particularly valuable in reinforcement learning settings.

[3] For the same setting, it is possible to group the vertices differently. We arbitrarily select one.

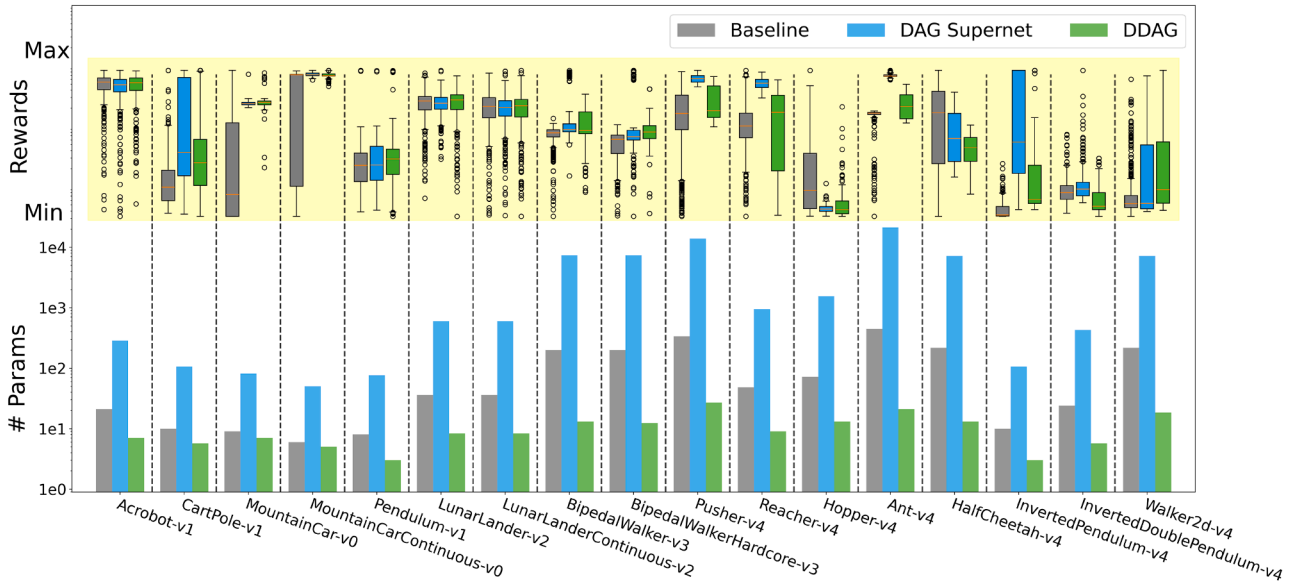

Fig. 3. Performance comparison of baseline architectures and those found by DAG-NAS (DAG supernets and DDAGs) in 17 RL environments. We attempt 100 trials in each environment and show the interquartile range (IQR) of achieved (normalized) rewards by boxplots (top figure). Also, the number of the architecture parameters is presented in a log scale (bottom figure). Overall, DDAGs perform well with a much smaller number of parameters, and in some cases, even outperform DAG supernets.

5. Experiments

We evaluate the proposed DAG-NAS framework on a range of reinforcement learning (RL) environments from Gymnasium using actor–critic PPO agents. PPO supports both discrete and continuous action spaces through a unified update rule and therefore requires no substantial modification across tasks. In our setting, policy learning and architecture search are performed simultaneously. We adopt a three-layer DAG supernetwork to provide representational capacity comparable to that of standard multilayer perceptrons (MLPs). Overall, the results show that DAG-NAS discovers architectures that achieve competitive performance across diverse RL tasks while providing improved interpretability by revealing task-relevant input features.

Our implementation[4] of the actor–critic PPO algorithm is based on CleanRL (Huang et al., 2022). We separate the actor and critic networks from the original implementation; both networks use a simple two-layer architecture[5] composed of `nn.Linear` modules in PyTorch. The actor and critic have $a \times b + b$ and $a \times 1 + 1$ learnable parameters, respectively, where a and b denote the input and output dimensions. This architecture serves as the baseline throughout our experiments. Accordingly, the DAG supernetwork is designed such that the number of parameters in the resulting DDAG does not exceed that of the baseline. All experiments were conducted on a single NVIDIA A6000 GPU. For each environment, architecture search and training required approximately 5–30h of wall-clock time, depending on environment complexity. Detailed parameters and times are shown in Table 1.

We train the baseline networks for 10 million steps and apply the same training budget to the DAG supernetworks. Note that training the DAG supernetwork jointly optimizes both architecture parameters and network weights under the single-level optimization in (5). After training, we fix the weights of the selected architecture and evaluate the resulting policy over 100 episodes, reporting the average return as the final score. We do not fix random seeds during training; instead, we repeat the entire procedure five times and report results from the run achieving the highest final performance. Across these trials, we observe stable architectural outcomes and consistent evaluation results.

Our experiments span a total of 17 RL environments, including Classic Control (Acrobot, CartPole, MountainCar, MountainCarContinuous, Pendulum), Box2D (LunarLander, LunarLanderContinuous, BipedalWalker, BipedalWalkerHardcore), and MuJoCo (Pusher, Reacher, Hopper, Ant, HalfCheetah, InvertedDoublePendulum, InvertedPendulum, Walker2d). Together, these environments cover a broad range of RL settings, encompassing discrete and continuous action spaces, varying state dimensionalities, and diverse control objectives such as stabilization, locomotion, and manipulation. This diversity allows us to assess the generality of DAG-NAS across representative RL scenarios commonly used in the literature.

In the following sections, we compare the performance of the baseline, the DAG supernetwork, and the final DDAG architectures. We further analyze the explainability of the discovered architectures and examine the impact of architectural choices on sample efficiency. Additional details on environment versions, hyperparameters, reward statistics, and visualizations of the searched architectures are provided in the supplementary material.

5.1. Performance comparison

Across 17 reinforcement learning environments, we evaluate the performance of baseline architectures, DAG supernetworks obtained after differentiable search, and the final Discrete DAG (DDAG) produced by DAG-NAG. For each environment, we evaluate trained actor-critic networks over 100 episodes and collect the corresponding cumulative rewards.

Fig. 3 summarizes the results by jointly presenting reward distributions and model sizes. The top panel shows the interquartile range (IQR) of normalized rewards, while the bottom panel reports the number of learnable parameters on the logarithmic scale. Taken together, these visualizations highlight the relationship between performance and parameter efficiency, clarifying their inherent trade-off.

Across most environments, DDAGs achieve performance comparable to or better than the baseline architectures while using substantially fewer parameters. In particular, in several classic control and Box2D tasks, DDAGs have an order of magnitude fewer parameters than the

[4] The full implementation of DAG-NAS, along with all experimental scripts, is publicly available at https://github.com/antaegun20/DAG-NAS.

[5] Including the input layer.

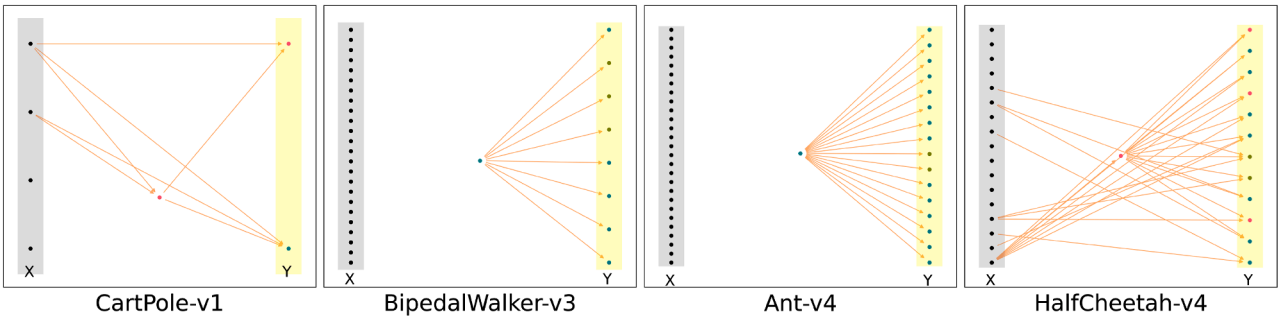

Fig. 4. DDAG actor architectures. The left vertices (shaded in gray) and right vertices (shaded in yellow) represent the input and output layers, respectively.

corresponding baseline architectures, but they achieve comparable or even better reward performance. This indicates that the architectures discovered by DAG-NAS are not merely compact, but also effective in preserving task-relevant computation.

DAG supernetworks often achieve strong performance, but this comes at the expense of a substantially larger number of parameters. This contrast highlights the importance of the discretization and pruning process: by removing redundant connections and operations, DDAG retains the core structural components required for high reward while removing unnecessary complexity. In some environments, such as Pendulum, DDAG even outperforms the corresponding supernetwork, suggesting that architectural simplification can improve generalization by reducing overparameterization.

In more challenging continuous control tasks, including several MuJoCo environments, performance varies across methods. While the baseline occasionally achieves higher peak rewards, DDAG consistently maintains competitive performance relative to its substantially smaller model size. These results emphasize that DAG-NAS discovers architectures that lie on a favorable performance-parameter efficiency frontier, achieving strong rewards with significantly reduced network complexity.

In addition, we observe that both the DAG supernetworks and resulting DDAGs exhibit reduced variability across repeated trials. As shown in Fig. 3, their reward distributions typically present narrower interquartile ranges (IQR) and fewer extreme outliers compared to their corresponding baselines. This indicates that the architectures discovered by DAG-NAS lead to more stable training dynamics and consistent policy performance across runs. Such stability suggests that the learned connectivity structures are less sensitive to initialization and optimization noise, contributing to robust behavior during training and evaluation.

Overall, Fig. 3 demonstrates that DAG-NAS effectively balances the performance and parameter efficiency across a diverse set of RL environments. The discovered DDAGs achieve competitive rewards while being substantially more compact than conventional multilayer perceptron baselines, validating parameter efficiency as a key empirical outcome of the proposed framework.

5.2. Architecture analysis

In this section, we analyze the architectures discovered by DAG-NAS and show how the learned DDAGs provide interpretable insights into RL policies.

5.2.1. Qualitative perspective

Fig. 4 presents typical DDAG actor architectures for some environments, including CartPole (Classic Control), BipedalWalker (Box2D), Ant, and InvertedPendulum (MuJoCo). In each diagram, vertices represent scalar-level operations and directed edges indicate information flow from state inputs to action outputs. We can observe that DDAGs explicitly expose which state variables are utilized for decision making. For example, in the CartPole environment, the state consists of cart position, cart velocity, pole angle, and pole angular velocity. The learned DDAG actor consistently relies only on the pole angle and angular velocity, discarding the remaining inputs. This aligns well with domain intuition and demonstrates that DAG-NAS implicitly performs feature selection during architecture search.

Similar patterns are observed across other environments. In LunarLander and LunarLanderContinuous, which share identical state representations but differ in action spaces, the discovered DDAG architectures are structurally similar. This suggests that DAG-NAS identifies task-relevant features primarily based on environment dynamics rather than action parametrization. Likewise, the DDAG actors for BipedalWalker and BipedalWalkerHardcore exhibit closely related structures, despite differences in terrain difficulty.

In several MuJoCo environments, such as Ant, Pusher, and Reacher, the learned DDAG actors exhibit minimal or no dependence on state inputs, producing nearly constant action parameters. This behavior indicates that high rewards can be achieved through stable control strategies in these tasks, and the resulting architectures make this property explicit. In contrast, environments such as Hopper and HalfCheetah retain connections to specific state variables, reflecting the need for more responsive control policies.

Overall, these qualitative observations demonstrate that DDAGs offer transparent, human-readable representations of learned policies. By visualizing which inputs influence decisions and how information propagates through the network, DAG-NAS enables practitioners to inspect, debug, and simplify reinforcement learning models in ways that are difficult with conventional multilayer perceptrons.

5.2.2. Quantitative perspective

Beyond qualitative visual interpretations, we conduct a quantitative structural analysis to provide empirical support for the interpretability of DDAG. Specifically, we employ architecture-level metrics to objectively characterize the extent to which the learned models selectively utilize state information.

To this end, we analyze the final DDAG actor and critic architectures in each environment using the following quantitative metrics:

- *Input Utilization*: the fraction of state input dimensions that have at least one active path to the output layer.
- *Connectivity Sparsity*: the ratio of active edges to the total number of possible edges in the corresponding scalar-level DDAG.
- *Stability Across Runs*: the consistency of selected input dimensions across independent architecture search trials, measured by the overlap ratio between runs.

Table 1 summarizes these metrics across the evaluated environments. In most tasks, DDAGs utilize only a small subset of the available state inputs, indicating strong feature selectivity. In several MuJoCo environments, it shows minimal usage dependence on state inputs, suggesting that stable control behaviors can emerge from relatively simple action distributions.

Table 1
Quantitative analysis.

Environment	Search Space		Input Utilization		Connectivity Sparsity (%)		Search
	Actor	Critic	Actor	Critic	Actor	Critic	Time
Acrobot-v1	3^{24}	3^{7}	2/6	2/6	3.33	10.42	8h
CartPole-v1	3^{10}	3^{3}	2/4	2/4	12.5	16.67	6h
MountainCar-v0	3^{9}	3^{4}	1/2	1/2	19.44	37.5	5h
MountainCarContinuous-v0	3^{6}	3^{7}	2/2	1/2	40.00	37.5	4h
Pendulum-v1	3^{9}	3^{7}	1/3	2/3	13.89	26.67	5h
LunarLander-v2	3^{72}	3^{9}	3/8	1/8	1.01	3.75	12h
LunarLanderContinuous-v2	3^{72}	3^{9}	2/8	2/8	0.83	5.0	12h
BipedalWalker-v3	3^{216}	3^{25}	0/24	1/24	0.13	0.48	49h
BipedalWalkerHardcore-v3	3^{216}	3^{25}	0/24	1/24	0.13	0.48	49h
Pusher-v4	3^{345}	3^{24}	0/23	1/23	0.11	0.52	76h
Reacher-v4	3^{55}	3^{12}	0/11	1/11	0.57	2.1	17h
Hopper-v4	3^{77}	3^{12}	1/11	1/11	1.01	2.1	27h
Ant-v4	3^{459}	3^{18}	0/27	1/27	0.08	0.38	104h
HalfCheetah-v4	3^{221}	3^{18}	7/17	4/17	0.46	4.95	52h
InvertedDoublePendulum-v4	3^{33}	3^{12}	1/11	2/11	1.3	3.5	74h
InvertedPendulum-v4	3^{12}	3^{5}	2/4	1/4	14.29	4.17	4h
Walker2d-v4	3^{221}	3^{18}	0/17	1/17	0.2	0.93	50h

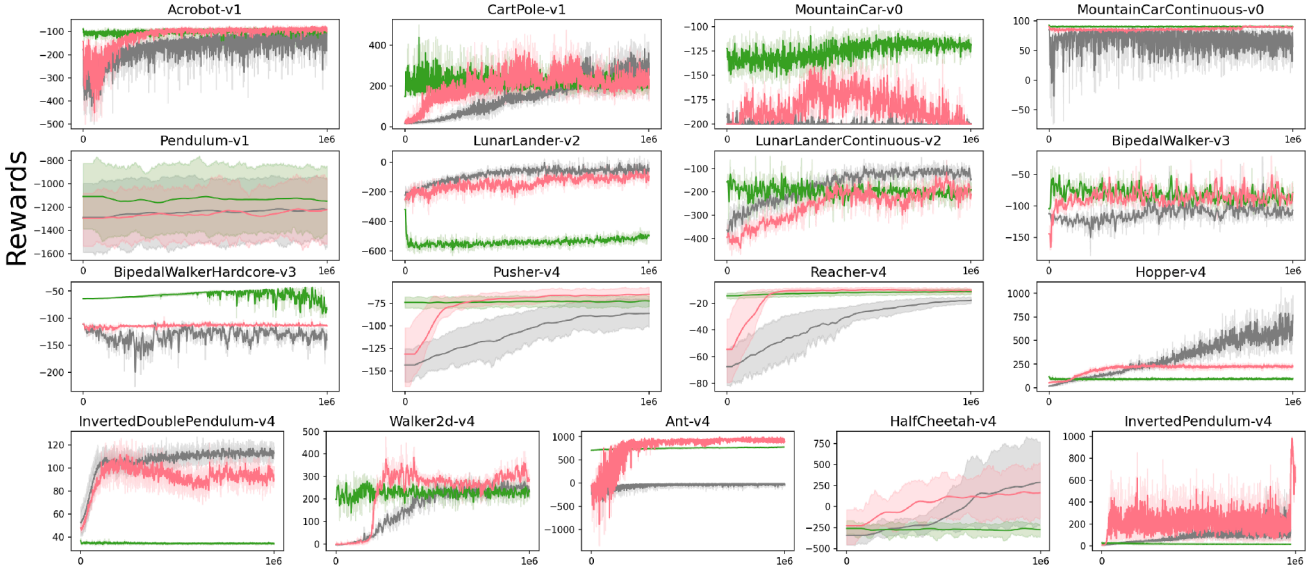

Fig. 5. Reward traces for $1M$ steps for DDAG (green), DDAG-RW (red), and the baseline (gray).

When state variables are actively used, connectivity sparsity remains high, particularly in Box2D and MuJoCo environments, and moderately lower in certain Classic Control tasks. This demonstrates that the pruning process effectively eliminates redundant connections while preserving task-relevant information. Overall, these results confirm that DDAG architectures are both compact and structurally selective across diverse reinforcement learning settings.

Moreover, the selected input subsets remain highly consistent across repeated searches, indicating that DAG-NAS converges to stable and interpretable architectural patterns rather than relying on incidental correlations. Combined with the qualitative analyses in Section 5.2.1, these results demonstrate that the structural properties identified by DAG-NAS are systematic and reproducible. This consistency is also reflected not only in the compactness of the resulting DDAG architectures, which contain significantly fewer parameters, but also in their stable performance across trials with reduced reward variance as presented in Fig. 3

5.3. Architecture impact on sample efficiency

In the previous sections, we set the weights $\mathbf{w}$ of DDAG to be the same as those of the DAG supernet, allowing us to bypass the typical weight-retraining process of NAS and directly evaluate the neural network model $(\alpha^*, \mathbf{w}^*)$. In this section, we further focus on architectural superiority. To this end, we initialize the weights of DDAG at random and then retrain them, which is denoted by DDAG-RW. By doing this, we aim to demonstrate that DDAG excels not only in achieving high rewards but also in rapidly acquiring knowledge, highlighting its significant contribution to sample efficiency.

We compare the training performance of the baseline, DDAG, and DDAG-RW over 1 million steps. We conduct 5 trials for each experiment and report the mean and standard error of rewards with a rolling window of 20. Fig. 5 shows their learning curves in 17 environments. Typically, DDAG starts strong and maintains its performance throughout training, but sometimes its performance declines as training continues. In contrast, DDAG-RW initially exhibits lower performance yet eventually matches or even surpasses DDAG. Compared to the baseline, DDAG-RW attains higher rewards in fewer steps, likely due to effective feature selection and fewer parameters. However, there are a few cases, such as Hopper, where baseline outperforms both DDAG and DDAG-RW. We observe that their corresponding DAG supernet also underperforms the baseline, indicating an unsuccessful architecture search.

6. Conclusion

We develop a fully differentiable NAS framework for reinforcement learning through scalar-level DAG modeling of neural networks. By simplifying cell design, explicitly modeling connectivity, and relaxing discrete architectural constraints, we construct a scalar-level DAG supernetwork that enables efficient architecture search. We further introduce a correlation-based pruning strategy to derive a Discrete DAG (DDAG) architecture with significantly fewer parameters. In addition, we eliminate the conventional weight-retraining step in NAS, making the overall search process more practical.

Through extensive evaluations on diverse RL benchmarks, we demonstrate the effectiveness and flexibility of DAG-NAS. The discovered DDAG architectures achieve competitive rewards despite their compact sizes and provide interpretable structures that explicitly highlight task-relevant input features. We further verify the architectural advantages of DDAGs in terms of sample efficiency, where they often exhibit faster learning compared to baseline architectures.

Beyond benchmark environments, the structural sparsity and interpretability of DDAGs suggest that DAG-NAS is well suited for practical control and robotics settings, where understanding sensor relevance and reducing model complexity are important considerations. While this work focuses on standard RL benchmarks, these properties indicate strong potential for deployment in real-world control scenarios.

For future work, an important direction is to extend the DAG-NAS framework to relax the requirement of pre-defining the depth and width of the DAG supernetwork while preserving flexibility and interpretability. Developing strategies to further reduce the computational cost of DAG-NAS also remains an important challenge for scaling the method to more complex tasks.

CRediT authorship contribution statement

Taegun An: Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Changhee Joo:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work is in part supported by IITP grant funded by the Korea government(MSIT) (RS-2024-00405128, 33%), in part by NRF grant funded by the Korea government(MSIT) (RS-2022-NR070834, 33%), and in part by the IITP-ICT Creative Consilience program grant funded by the Korea government(MSIT) (IITP-2026-RS-2020-II201819, 33%).

Supplementary material

Supplementary material associated with this article can be found in the online version at 10.1016/j.neunet.2026.108901.

Appendix A. Technical appendices and supplementary material

1. Hyperparameters We slightly modified the reliable PPO implementation of CleanRL (Huang et al., 2022). We use the PPO parameters shown in Table A.1. The same hyperparameters are used for the baseline, DAG supernet, and DDAG, across all the 17 environments. Those values are also publicly available with our DAG-NAS implementation repository: https://github.com/antaegun20/DAG-NAS.

Table A.1
Hyperparameters of PPO algorithm.

Name	notation	value
gamma	γ	0.99
GAE Lambda	λ	0.95
Value function coefficient	c_{vf}	0.5
Entropy coefficient	c_{ent}	0
Clipping coefficient	c_{vf}	0.2
Normalized advantage	adv_norm	True
Clip value loss	cvl	True
Target KL divergence	KL_{target}	None
max grad norm	c_{vf}	0.5
update epochs	K	4
rollout steps	T	512
num minibatches	mB	8
num envs	–	4
num steps	–	1,000,000
Learning rate	–	0.0002
Learning rate annealing	–	False
beta 1	β_1	0.9
beta 2	β_2	0.999

2. Rewards The rewards of Fig. 3 in the main paper are min-max normalized. Table A.2 shows the mean and standard deviation of unnormalized rewards.

3. DDAG Architectures Searched by DAG-NAS In this section, we report the architectures of actor and critic networks, found in 17 environments. The left subfigure shows the architecture of the actor network, and the right subfigure shows the architecture of the critic network. In each figure, we list the input features on the left side using the notation $x_1, \ldots, x_a$ and output features on the right side as $y_1, \ldots, y_b$. Note that in continuous control tasks, $y_1, \ldots, y_{b/2}$ is for the mean value of actions, and $y_{b/2+1}, \ldots, y_b$ is for the standard deviation of the actions. We present the vertex with the discrete operation of $Tanh$ as pink, $LeakyReLU$ as olive green, and $Linear$ as mint. Figs. A.1–A.17

4. DAG-NAS with Deeper Networks In addition, we present the result of DAG-NAS with deeper networks on Hopper-v4. We construct DDAG with 5 layers including an input layer where each layer has a configuration of $[11, 88, 176, 352, 6]$, and name it as DAG-L. We trained DAG-L for 1M steps, produced DDAG-L, and presented its architecture in Fig. A.18. Also, we compare the number of parameters and performances in Fig. A.19. The result shows that our DAG framework can be extended to deeper and larger networks.

4. Extension to image classification We have MNIST classification results with an accuracy of 96.9%. Fig. A.20 shows the points in the input image where the connections to the output vertices are activated. For example, for the image of class 0, the activated connections of the 0th node are visualized as yellow dots on the image. The DAG-NAS framework is applicable to images and works as a feature selector as described in the main paper. As we intended to focus on RL, it is omitted. It took less than 0.1 GPU days for MNIST, for 100 search epochs.

5. Maturity of DAG-NAS Fig. A.21 shows the evolution of rewards of DAG-supernetwork and derived DDAG with respect to the RL updates. It shows that performant DDAG is determined in a relatively earlier stage of search and remains stable for the rest of the period. As the search progresses, the performance of DAG-supernetwork eventually comes close to DDAG performance, which implies that the connectivity and softmax weights become similar to DDAG.

Table A.2
Mean and standard deviation of rewards, obtained from 100 trials in each environment.

Environment	DAG	DDAG	Baseline
Acrobot-v1	–89.52 ± 34.50	–86.96 ± 26.64	–87.17 ± 27.02
CartPole-v1	261.98 ± 166.36	193.68 ± 132.65	125.44 ± 82.17
MountainCar-v0	–111.49 ± 2.79	–111.34 ± 8.51	–167.33 ± 34.33
MountainCarContinuous-v0	91.45 ± 1.47	90.85 ± 2.13	49.46 ± 51.89
Pendulum-v1	–1241.07 ± 277.03	–1231.57 ± 300.03	–1284.92 ± 273.85
LunarLander-v2	–179.92 ± 129.61	–188.18 ± 208.19	–185.98 ± 188.34
LunarLanderContinuous-v2	–207.2 ± 188.34	–210.22 ± 211.84	–205.78 ± 204.88
BipedalWalker-v3	–101.77 ± 31.65	–105.0 ± 26.61	–123.24 ± 22.28
BipedalWalkerHardcore-v3	–96.06 ± 43.17	–99.82 ± 23.01	–126.86 ± 25.73
Pusher-v4	–55.25 ± 7.73	–104.29 ± 32.91	–135.57 ± 73.18
Reacher-v4	–11.48 ± 2.23	–31.25 ± 18.50	–31.15 ± 12.90
Hopper-v4	73.85 ± 38.8	131.05 ±171.37	353.88 ± 334.47
Ant-v4	955.53 ± 42.11	148.08 ±391.87	–127.15 ± 334.73
HalfCheetah-v4	146.91 ± 330.88	–25.17 ± 215.17	323.22 ± 521.49
InvertedDoublePendulum-v4	576.25 ± 354.48	279.68 ± 295.53	39.06 ± 50.6
InvertedPendulum-v4	110.0 ± 39.82	67.72 ± 33.22	93.36 ± 34.27
Walker2d-v4	113.01 ± 136.32	134.53 ±124.1	55.3 ± 88.82

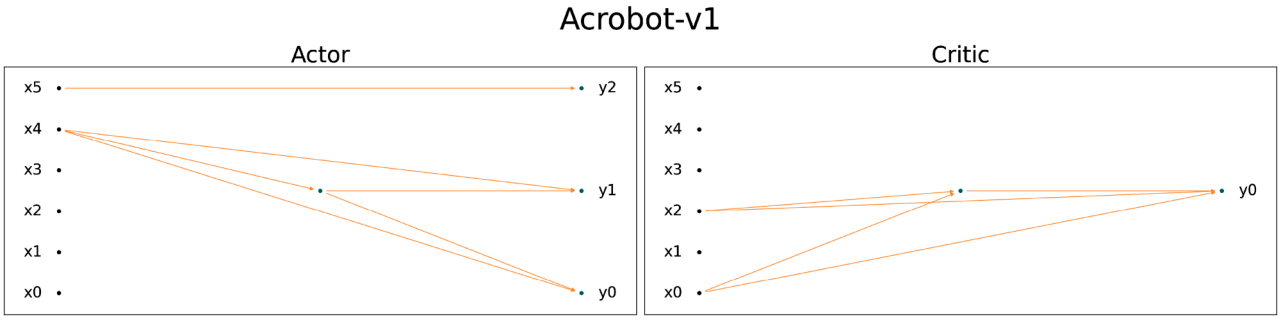

Fig. A.1. Architectures of actor and critic networks for the Acrobot-v1 environment.

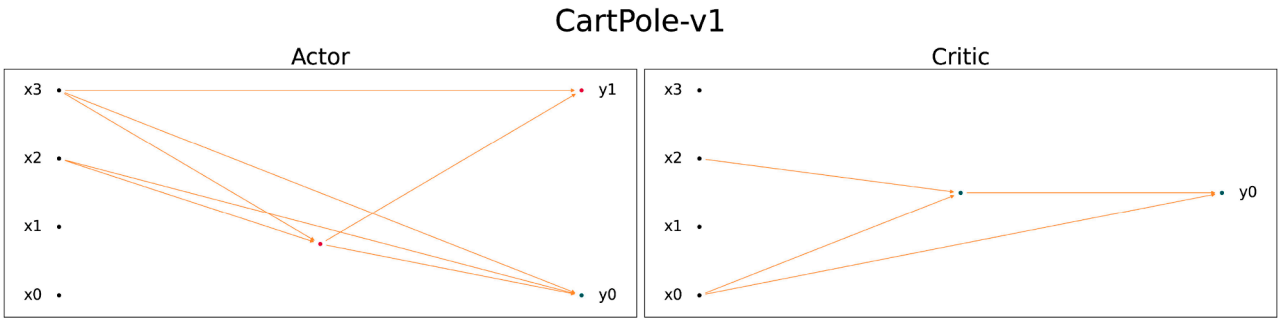

Fig. A.2. Architectures of actor and critic networks for the CartPole-v1 environment.

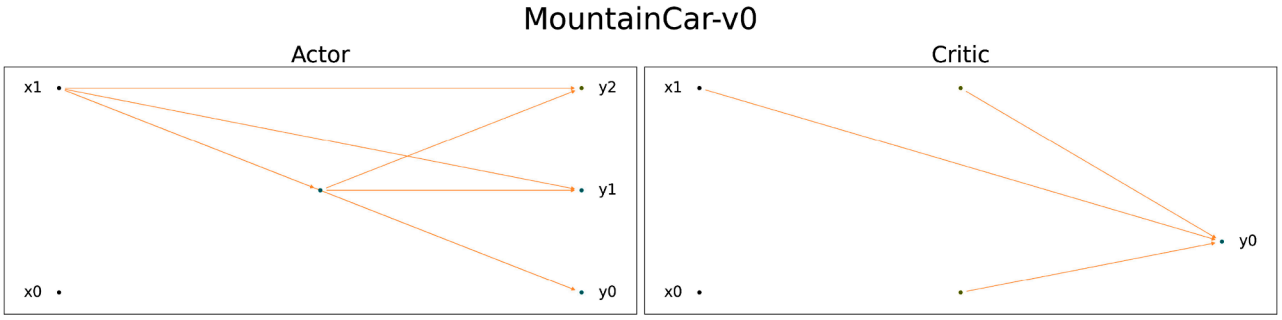

Fig. A.3. Architectures of actor and critic networks for the MountainCar-v1 environment.

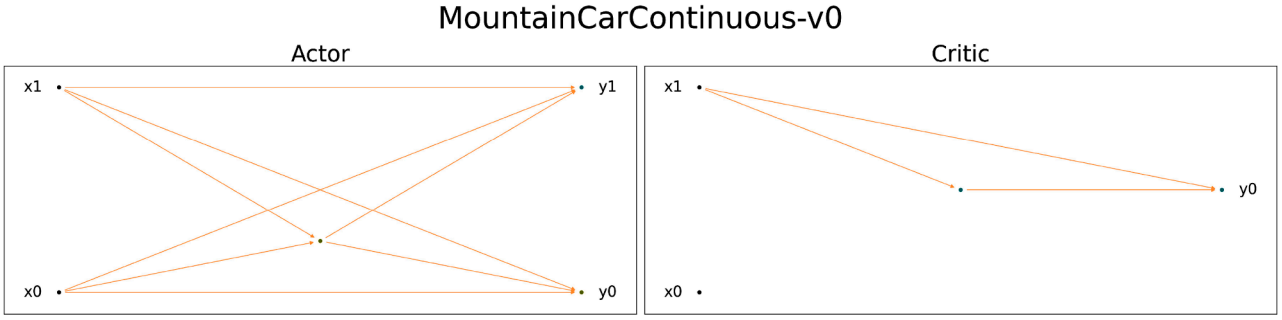

Fig. A.4. Architectures of actor and critic networks for the MountainCarContinuous-v1 environment.

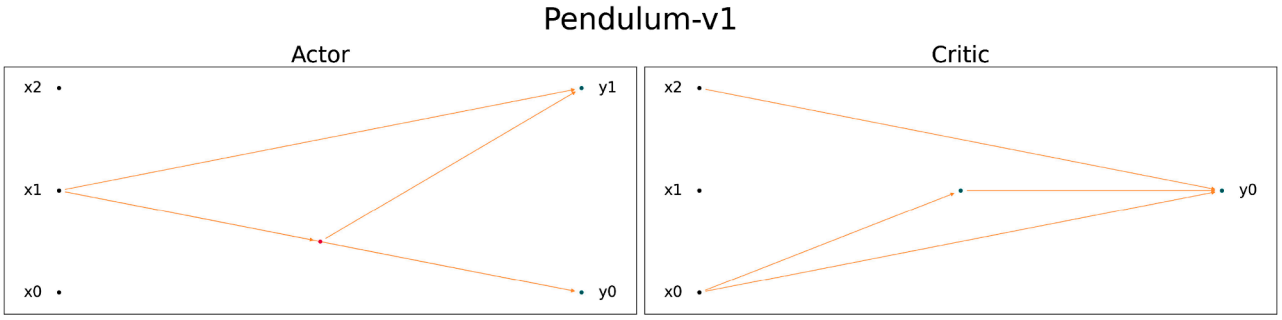

Fig. A.5. Architectures of actor and critic networks for the Pendulum-v1 environment.

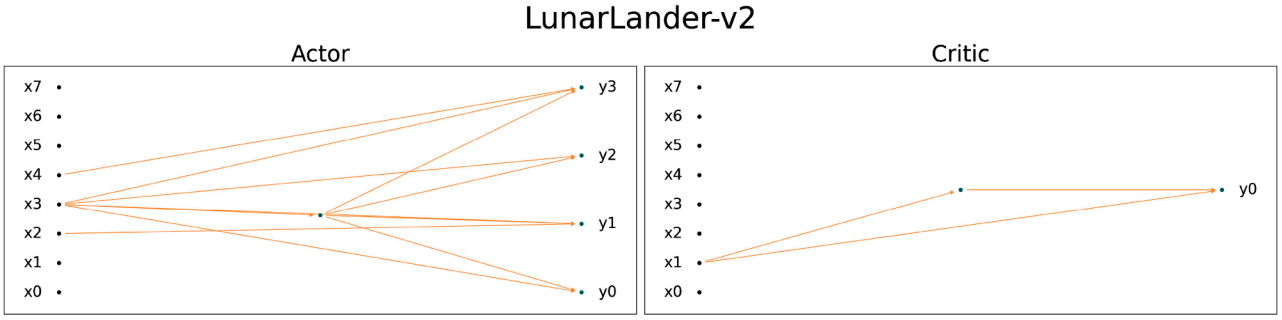

Fig. A.6. Architectures of actor and critic networks for the LunarLander-v2 environment.

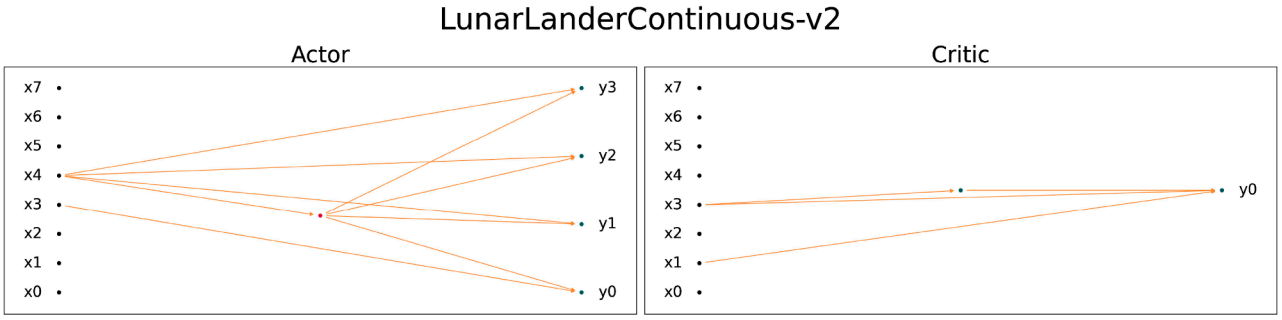

Fig. A.7. Architectures of actor and critic networks for the LunarLanderContinuous-v2 environment.

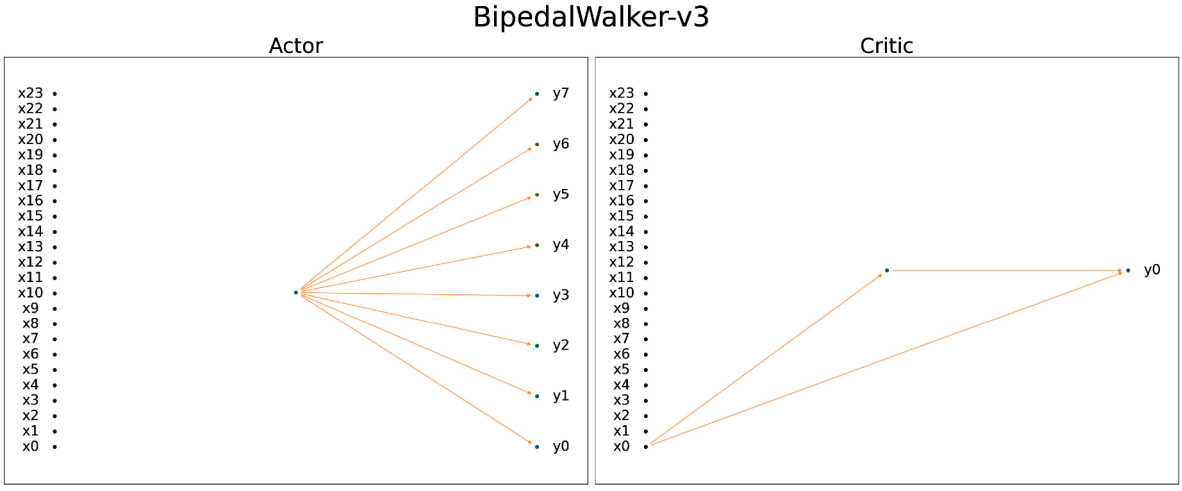

Fig. A.8. Architectures of actor and critic networks for the BipedalWalker-v3 environment.

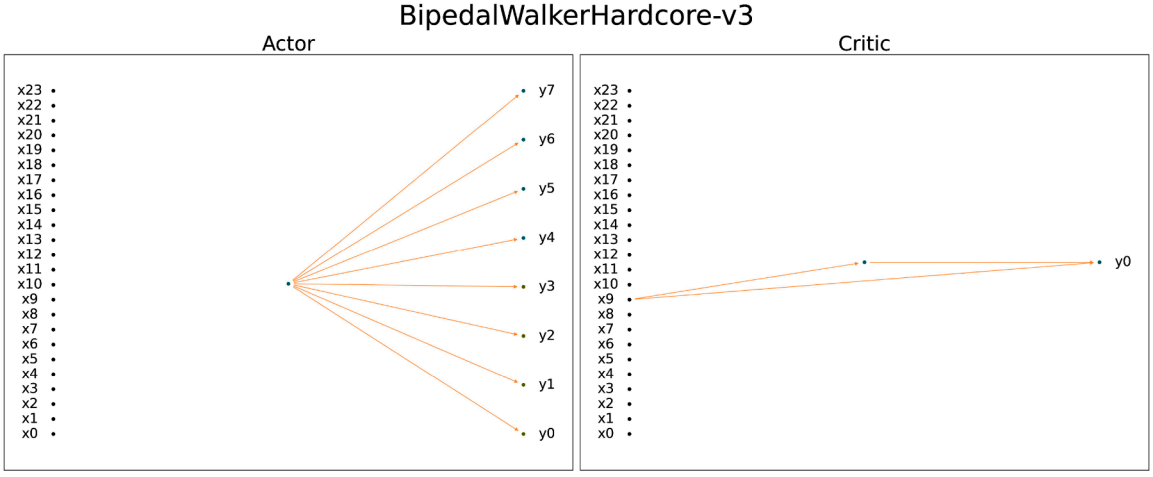

Fig. A.9. Architectures of actor and critic networks for the BipedalWalkerHardcore-v4 environment.

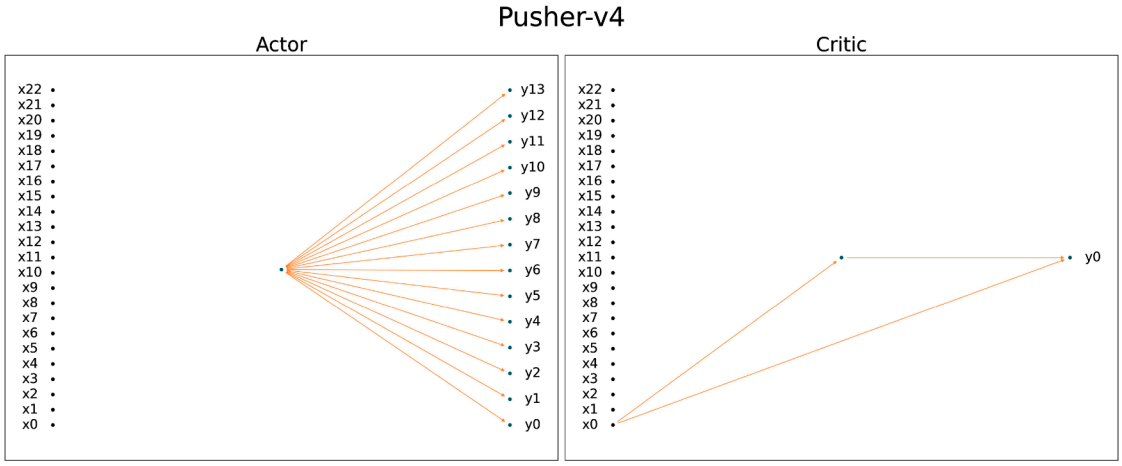

Fig. A.10. Architectures of actor and critic networks for the Pusher-v4 environment.

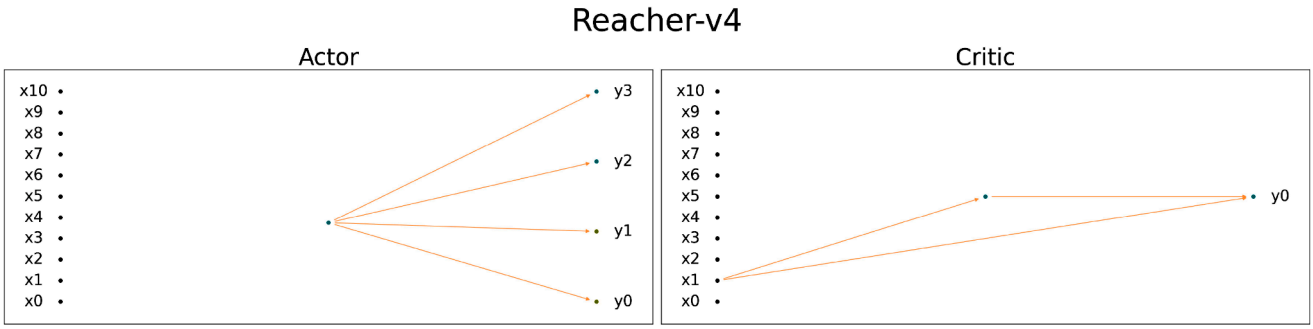

Fig. A.11. Architectures of actor and critic networks for the Reacher-v4 environment.

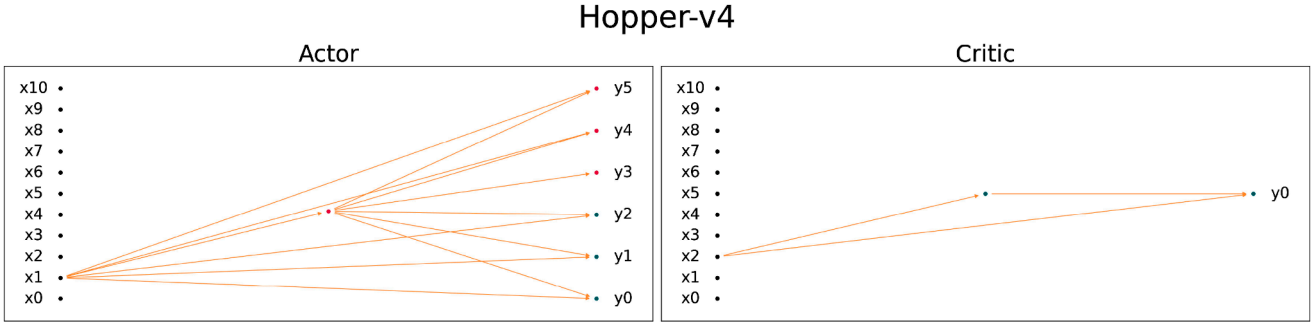

Fig. A.12. Architectures of actor and critic networks for the Hopper-v4 environment.

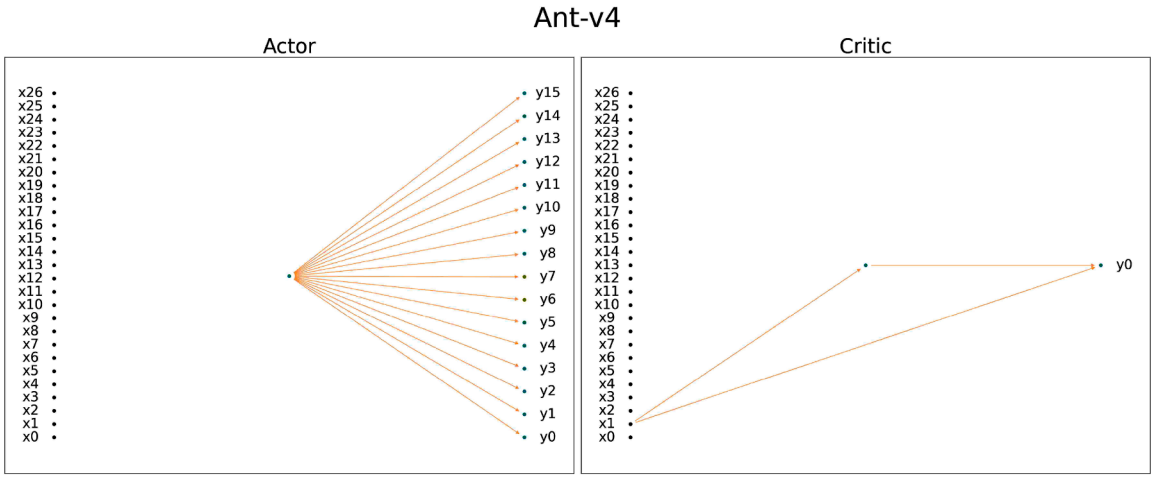

Fig. A.13. Architectures of actor and critic networks for the Ant-v4 environment.

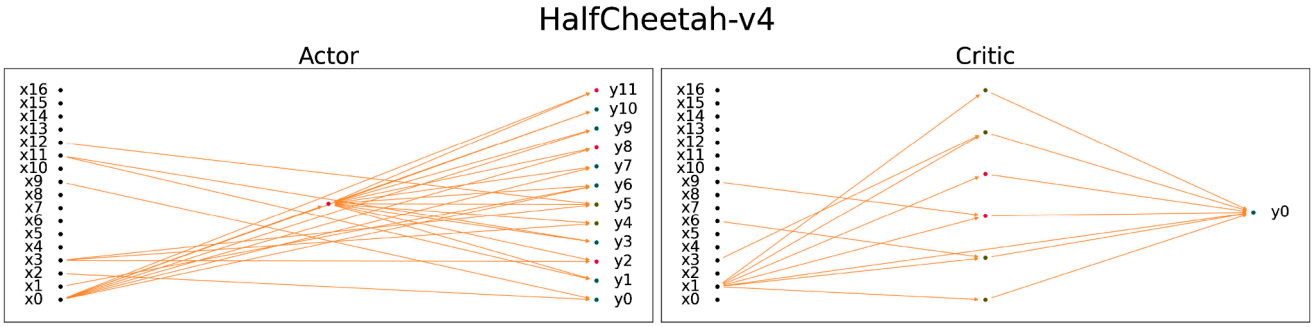

Fig. A.14. Architectures of actor and critic networks for the HalfCheetah-v4 environment.

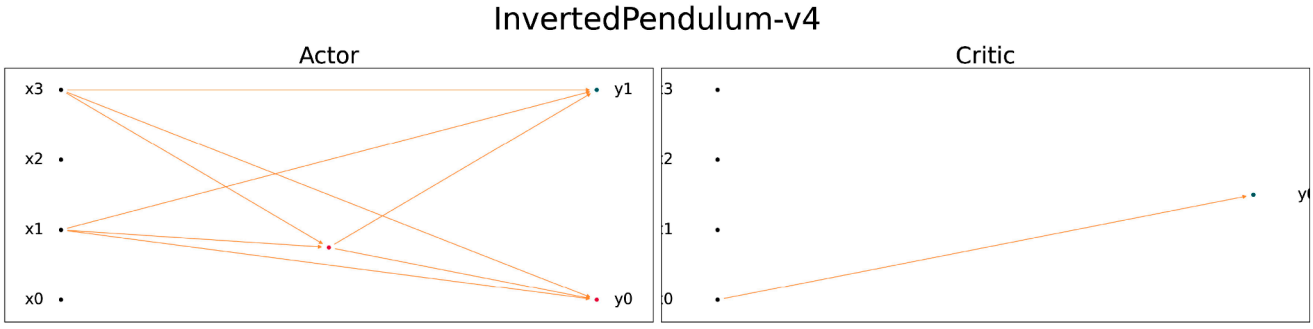

Fig. A.15. Architectures of actor and critic networks for the InvertedPendulum-v4 environment.

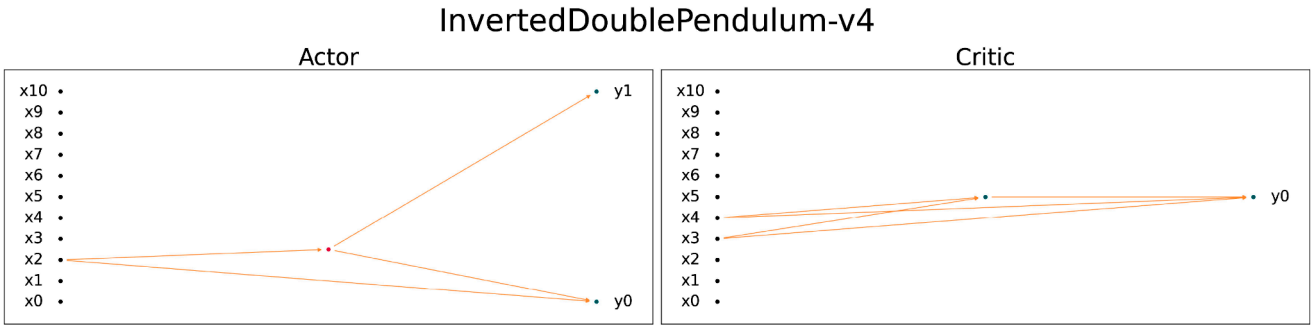

Fig. A.16. Architectures of actor and critic networks for the InvertedDoublePendulum-v4 environment.

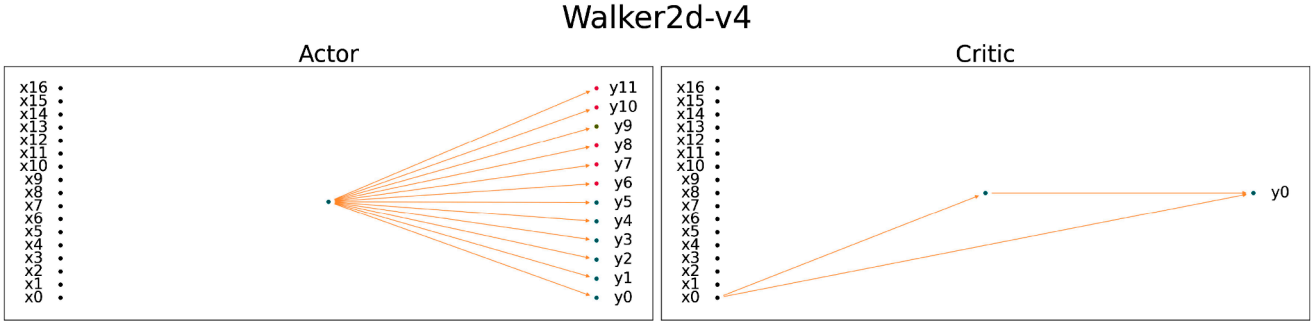

Fig. A.17. Architectures of actor and critic networks for the Walker2d-v4 environment.

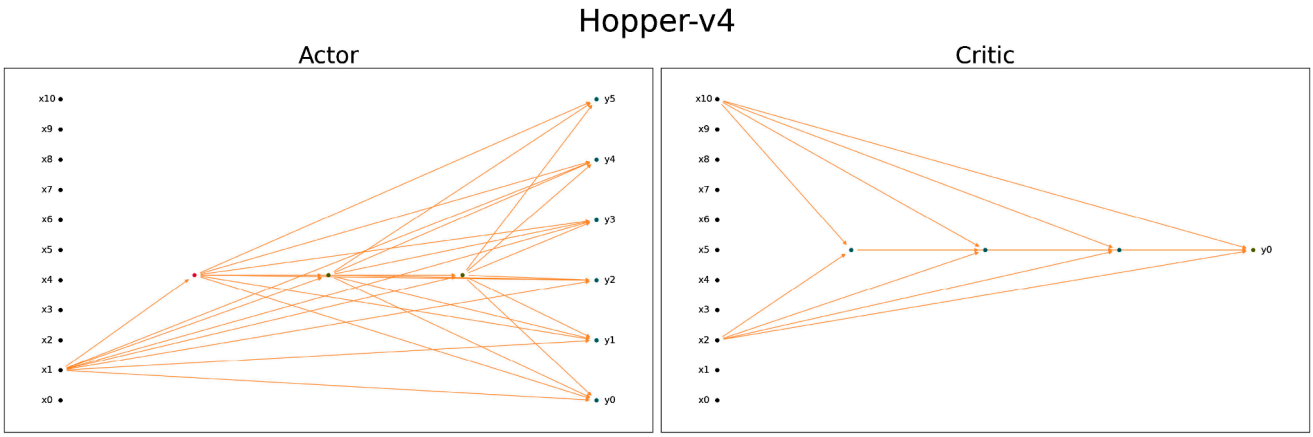

Fig. A.18. Architectures of DDAG-L actor and critic networks for the Hopper-v4 environment.

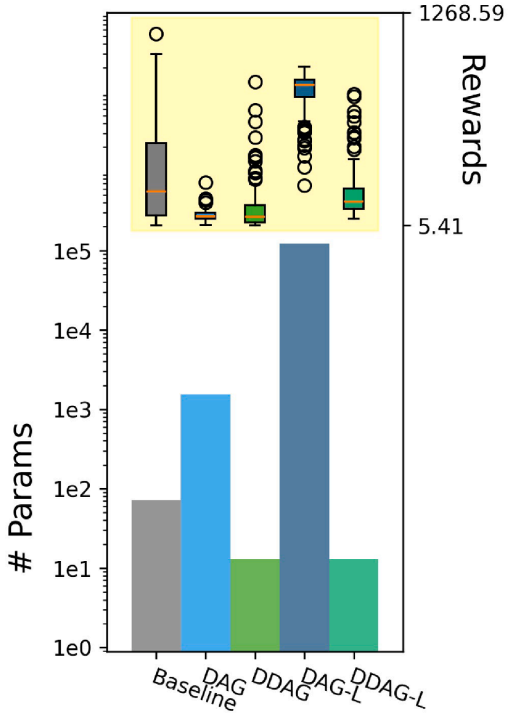

Fig. A.19. Rewards from 100 episodes in the Hopper-v4 environment.

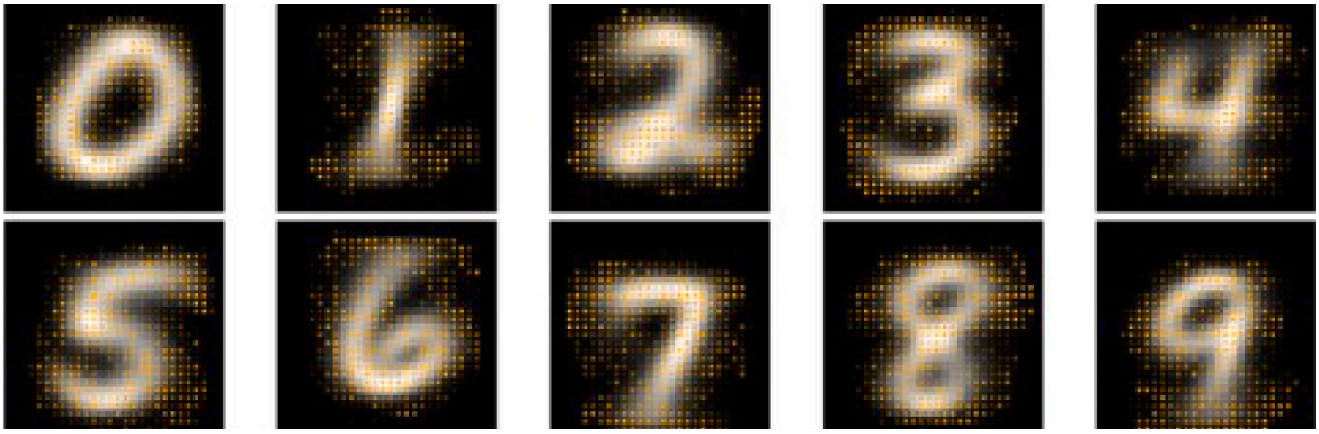

Fig. A.20. DAG-NAS for MNIST Classification.

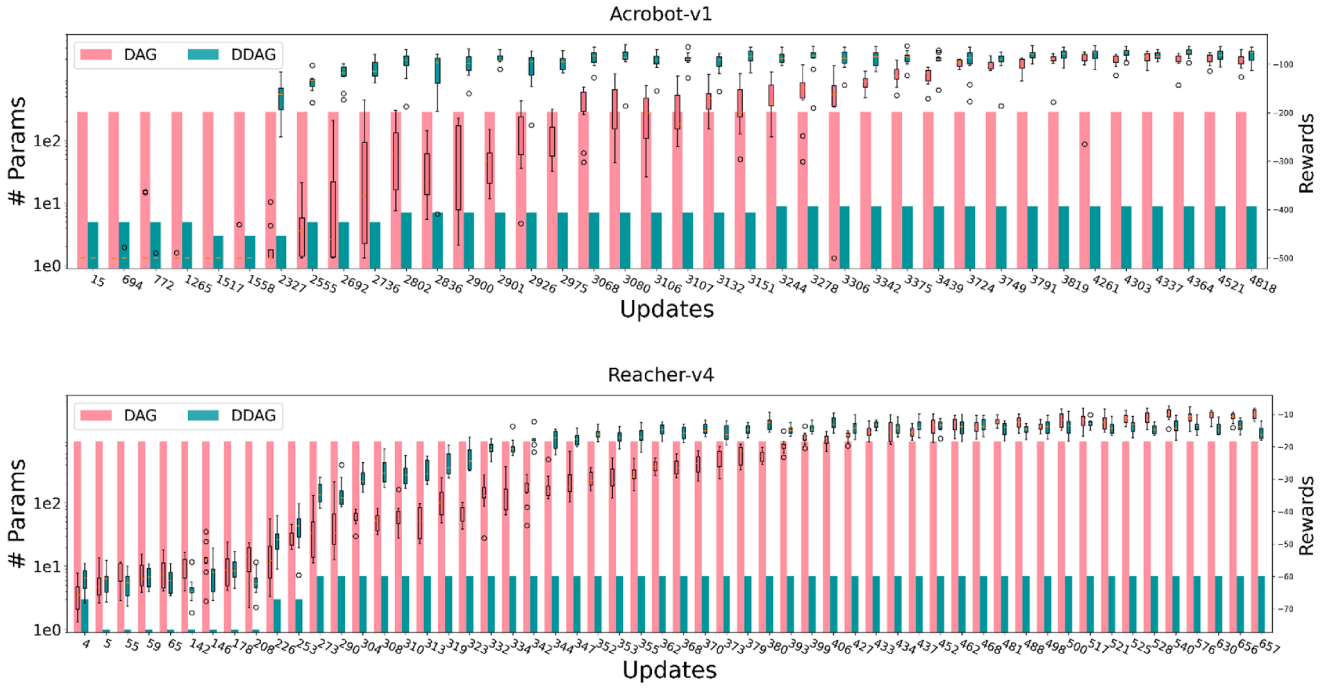

Fig. A.21. Evolution of rewards of DAG/DDAG in Acrobot-v1 and Reacher-v4 environments.

References

Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., & Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. In A. Beygelzimer, Y. Dauphin, P. Liang, & J. W. Vaughan (Eds.), *Advances in neural information processing systems*.

An, T., & Joo, C. (2024). CycleGANAS: Differentiable neural architecture search for cycleGAN. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR) workshops* (pp. 1655–1664).

Carmichael, Z. J., Moon, T., & Jacobs, S. A. (2023). Learning debuggable models through multi-objective NAS. In *AutoML conference 2023*.

Chan, S. C. Y., Fishman, S., Korattikara, A., Canny, J., & Guadarrama, S. (2020). Measuring the reliability of reinforcement learning algorithms. In *International conference on learning representations*.

Ci, Y., Lin, C., Sun, M., Chen, B., Zhang, H., & Ouyang, W. (2021). Evolving search space for neural architecture search. arXiv:2011.10904.

Elsken, T., Metzen, J. H., & Hutter, F. (2019). Neural architecture search: A survey. *Journal of Machine Learning Research, 20*(55), 1–21.

Franke, J. K. H., Koehler, G., Biedenkapp, A., & Hutter, F. (2021). Sample-efficient automated deep reinforcement learning. In *International conference on learning representations*.

Gaier, A., & Ha, D. (2019). Weight agnostic neural networks. arXiv:1906.04358.

Geada, R., & McGough, A. S. (2022). Spidernet: Hybrid differentiable-evolutionary architecture search via train-free metrics. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR) workshops* (pp. 1962–1970).

Huang, S., Dossa, R. F. J., Ye, C., Braga, J., Chakraborty, D., Mehta, K., & Araújo, J. G. M. (2022). CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research, 23*(274), 1–18.

Kadra, A., Arango, S. P., & Grabocka, J. (2023). Breaking the paradox of explainable deep learning. arXiv:2305.13072.

Kang, J.-S., Kang, J., Kim, J.-J., Jeon, K.-W., Chung, H.-J., & Park, B.-H. (2023). Neural architecture search survey: A computer vision perspective. *Sensors, 23*(3), 1713.

Klyuchnikov, N., Trofimov, I., Artemova, E., Salnikov, M., Fedorov, M., & Burnaev, E. (2020). Nas-bench-NLP: Neural architecture search benchmark for natural language processing. arXiv:2006.07116.

Lee, H., Hyung, E., & Hwang, S. J. (2021). Rapid neural architecture search by learning to generate graphs from datasets. In *International conference on learning representations*.

Li, L., & Talwalkar, A. (2020). Random search and reproducibility for neural architecture search. In R. P. Adams, & V. Gogate (Eds.), *Proceedings of the 35th uncertainty in artificial intelligence conference* (pp. 367–377). PMLR (*vol. 115*). Proceedings of Machine Learning Research.

Liu, C., Zoph, B., Neumann, M., Shlens, J., Hua, W., Li, L.-J., Fei-Fei, L., Yuille, A., Huang, J., & Murphy, K. (2018). Progressive neural architecture search. In *Proceedings of the european conference on computer vision (ECCV)*.

Liu, H., Simonyan, K., & Yang, Y. (2019). DARTS: Differentiable architecture search. In *International conference on learning representations*.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems*. Curran Associates, Inc. (*vol. 30*).

Montavon, G., Lapuschkin, S., Binder, A., Samek, W., & Müller, K.-R. (2017). Explaining nonlinear classification decisions with deep taylor decomposition. *Pattern Recognition, 65*, 211–222. https://doi.org/10.1016/j.patcog.2016.11.008

Mysore, S., Mabsout, B. E., Mancuso, R., & Saenko, K. (2021). Honey. I shrunk the actor: A case study on preserving performance with smaller actors in actor-critic RL. In *2021 IEEE conference on games (cog)*. IEEE Press.

Parker-Holder, J., Rajan, R., Song, X., Biedenkapp, A., Miao, Y., Eimer, T., Zhang, B., Nguyen, V., Calandra, R., Faust, A., Hutter, F., & Lindauer, M. (2022a). Automated reinforcement learning (autoRL): A survey and open problems. *Journal of Artificial Intelligence Research, 74*, 517–568. https://doi.org/10.1613/jair.1.13596

Parker-Holder, J., Rajan, R., Song, X., Biedenkapp, A., Miao, Y., Eimer, T., Zhang, B., Nguyen, V., Calandra, R., Faust, A., Hutter, F., & Lindauer, M. (2022b). Automated reinforcement learning (autoRL): A survey and open problems. *Journal of Artificial Intelligence Research, 74*, 517–568. https://doi.org/10.1613/jair.1.13596

Ru, B., Wan, X., Dong, X., & Osborne, M. (2021). Interpretable neural architecture search via bayesian optimisation with weisfeiler-lehman kernels. In *International conference on learning representations*.

White, C., Neiswanger, W., Nolen, S., & Savani, Y. (2020). A study on encodings for neural architecture search. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, & H. Lin (Eds.), *Advances in neural information processing systems* (pp. 20309–20319). Curran Associates, Inc. (*vol. 33*).

White, C., Safari, M., Sukthanker, R., Ru, B., Elsken, T., Zela, A., Dey, D., & Hutter, F. (2023). Neural architecture search: Insights from 1000 papers. arXiv:2301.08727.

Xia, X., Xiao, X., Wang, X., & Zheng, M. (2022). Progressive automatic design of search space for one-shot neural architecture search. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision (WACV)* (pp. 2455–2464).

Xie, S., Zheng, H., Liu, C., & Lin, L. (2019). SNAS: Stochastic neural architecture search. In *International conference on learning representations*.

Zhou, D., Jin, X., Lian, X., Yang, L., Xue, Y., Hou, Q., & Feng, J. (2021). Autospace: Neural architecture search with less human interference. In *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)* (pp. 337–346).
Zoph, B., & Le, Q. (2017). Neural architecture search with reinforcement learning. In *International conference on learning representations*.

Taegun An is currently a PhD student at Korea University. He received BS degree in Computer Science from Ulsan National Institute of Science and Technology (UNIST). His research interests are reinforcement learning (RL) and neural architecture search (NAS).

Changhee Joo received the PhD degree from Seoul National University in 2005. He was with Purdue University and The Ohio State University, and then with Korea University of Technology and Education (KoreaTech) and Ulsan National Institute of Science and Technology (UNIST). Since 2019, he has been with Korea University. His research interests are in the broad areas of networking and machine learning. He was a recipient of the IEEE INFOCOM 2008 Best Paper Award, the KICS Haedong Young Scholar Award (2014), the ICTC 2015 Best Paper Award, and the GAMENETS 2018 Best Paper Award. He is an Associate Editor of the *IEEE Transactions Vehicular Technology*, a Division Editor of the *Journal of Communications and Networks*. He was a general co-chair of ACM MobiHoc 2022, and an Associate Editor of the *IEEE/ACM Transactions on Networking*. He also servered many primary conferences as a technical program committee member, including IEEE INFOCOM, ACM MobiHoc, IEEE WiOpt, and IEEE GLOBECOM.